CSE 344 — System Programming

Homework #3

Multi-Process Word Transportation
and Concurrent Sorting System

Due Date: 15/04/2026

Author: Ayşe Feyza SERBEST

ID: 220104004052

Contents

1. Introduction

This report describes the design, implementation, and testing of a multi-process word transportation and concurrent sorting system developed for CSE 344 System Programming, Homework #3. The system reads words from a text file, distributes them across the floors of a virtual building, transports individual characters between floors using elevator processes, and reconstructs each word on its designated sorting floor using concurrent sorting processes.

1.1 Overview of the System

The system models a building with F floors. Each floor hosts several types of worker processes. Words are decomposed into characters; each character is an independent transport task. The system terminates only when every word has been fully sorted and the output file has been written.

1.2 Main Challenges

- **Concurrency:** many processes read and modify the same shared-memory structures simultaneously, so every critical section must be protected.
- **Synchronization:** semaphores are used instead of busy-waiting to avoid wasting CPU cycles, and lock-ordering rules prevent deadlocks.
- **Process coordination:** the parent must detect when all words are complete and then perform a clean, ordered shutdown without zombie processes.
- **Duplicate characters:** words such as `apple` contain repeated letters; the sorting algorithm must not confuse the two `p` characters.
- **Elevator capacity:** both elevators have a finite load limit that must be enforced atomically to avoid overflow.

2. System Architecture

2.1 Process Types

Parent / Coordinator Process — reads the input file, initialises shared memory and all semaphores, forks every child process, monitors completion via a semaphore-protected `done_count`, writes the output file, prints the summary, and performs clean shutdown. All child processes are created exclusively by the parent.

Word-Carrier Processes — one or more per floor, fixed to their initial floor. They scan the word list in round-robin order, atomically claim an unclaimed word, then attempt all-or-nothing admission by locking both the arrival floor and the sorting floor simultaneously. On failure the word is released and a retry counter is incremented.

Letter-Carrier Processes — one or more per floor at startup; may migrate to other floors via the reposition elevator. They pick unclaimed character tasks on their current floor, use the delivery elevator if the destination differs, place the character in the first free slot of the sorting area, and then remain on the destination floor. If no work is found after several idle ticks they use the reposition elevator to move to a random floor. If a floor ever has zero live letter-carriers the parent spawns a fresh batch.

Sorting Processes — one or more per floor, never change floor. They continuously scan every word assigned to their floor, fix correctly-placed characters, move characters to empty target slots, and swap characters when the target is occupied but not fixed. A per-word mutex (`sem_trywait`) ensures only one sorting process operates on a given word at a time.

Delivery Elevator Process — a single process that moves UP and DOWN, serving requests in the current direction first. It reverses at the top/bottom floor or when no further requests exist in the current direction and its load is zero. Capacity is enforced atomically.

Reposition Elevator Process — identical movement policy to the delivery elevator, but used exclusively for relocating idle letter-carriers to random floors.

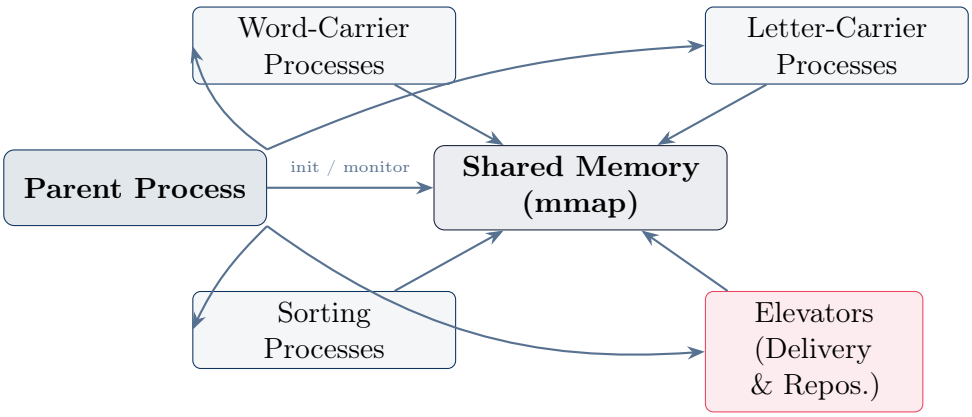
2.2 Process Interaction

All inter-process communication is done through a single anonymous shared-memory segment created with `mmap(MAP_SHARED|MAP_ANONYMOUS)`. There are no pipes or message queues. Semaphores embedded inside the shared-memory structures provide all synchronisation.

Process	Floor	Communicates With
Parent	—	All children via shared memory; `waitpid`
Word-Carrier	Fixed	`shm->words[]`, `shm->floors[]` (admission)
Letter-Carrier	Mobile	`shm->words[]` (tasks), delivery elevator
Sorting Process	Fixed	`shm->words[]` (sorting area)
Delivery Elevator	—	`shm->delivery_requests[]`, `shm->delivery`
Reposition Elevator	—	`shm->reposition_requests[]`, `shm->reposition`

Table 1: Process types and their communication targets

2.3 Architecture Diagram



3. Data Structures and Shared Memory Design

3.1 Shared Memory Layout

A single contiguous block is allocated with `mmap`. Pointers into this block are set by `shm_assign_pointers()` using a bump-allocator pattern so that every structure is naturally aligned. The layout in order is:

- `t_shm_header` — global counters and header semaphores
- `t_word[word_count]` — one entry per input word
- `t_floor[num_floors]` — per-floor capacity and mutex
- `t_elevator` × 2 — delivery and reposition elevator state
- `int[num_floors]` × 2 — `delivery_requests` and `reposition_requests` arrays
- `t_lc_state[num_floors × lc_per_floor]` — letter-carrier tracking

3.2 t_shm_header

Contains `total_words`, `rr_index` (round-robin cursor), `admitted_count`, `done_count`, `shutdown` flag, `retry_count`, `delivery_ops`, `reposition_ops`, and three semaphores: `rr_mutex` (protects `rr_index`), `done_mutex` (protects `done_count` / `admitted_count`), and `stats_mutex` (protects the statistics counters).

3.3 t_word

The central data structure. Key fields:

- `original[MAX_WORD_LEN]` — the word as read from input, never modified
- `sorting_area[MAX_WORD_LEN]` — characters placed by letter-carriers
- `occupied[MAX_WORD_LEN]` — 1 if that slot holds a character
- `fixed[MAX_WORD_LEN]` — 1 if the character at that slot is correct
- `tasks[MAX_WORD_LEN]` — one `t_char_task` per character (`word_id`, `character`, `original_index`, `src_floor`, `dest_floor`, `claimed`, `delivered`)
- `claimed` — set to 1 by a word-carrier attempting admission
- `admitted` — set to 1 once both floors have capacity
- `completed` — set to 1 once all `fixed[]` entries are 1
- `word_mutex` — per-word semaphore; only one sorter may operate at a time

3.4 t_elevator

Holds `current_floor`, `direction` (`ELEV_UP=+1` / `ELEV_DOWN=-1`), `capacity`, `current_load`, a `mutex` semaphore, and a `request_sem` used to wake the elevator process when a new request arrives.

4. Synchronization Strategy

4.1 Semaphores Used

4.2 Race Condition Prevention

Every shared variable that can be written by more than one process is protected by a semaphore. The two most sensitive operations are:

- **Word claiming:** `rr_mutex` is held while both checking and setting `word->claimed`,

Semaphore	Init	Protects / Purpose
`header->rr_mutex`	1	`rr_index` round-robin cursor
`header->done_mutex`	1	`admitted_count`, `done_count`
`header->stats_mutex`	1	`retry_count`, `delivery_ops`, `reposition_ops`
`word->word_mutex`	1	Per-word sorting area (one sorter at a time)
`floor->mutex`	1	Per-floor `current_word_count`
`elevator->mutex`	1	Elevator state (floor, load, requests)
`elevator->request_sem`	0	Wakes elevator process on new request

Table 2: Semaphores and their roles

making the check-then-set atomic.

- **Floor capacity admission:** both floor mutexes are acquired together before checking or incrementing `current_word_count`. A consistent lock-ordering rule (always lock lower floor index first) prevents deadlock between processes that might target the same pair of floors.

4.3 Critical Sections

- `rr_pick()`: holds `rr_mutex` for index read and increment only — released before the scan loop.
- `try_admit()`: holds both floor mutexes together for the capacity check and increment. Released immediately after.
- `find_task()` / `place_char()`: `word_mutex` is held only for the claim or placement operation, not across I/O or sleeps.
- `sort_step()`: uses `sem_trywait` so a sorter skips a locked word rather than blocking, avoiding starvation of other words.

4.4 Deadlock Avoidance

The only place where two locks are held simultaneously is `try_admit()`, and the lock ordering (`lo = min(arrival, sorting)`, `hi = max(...)`) is always respected. No other code path acquires more than one mutex at a time.

5. Word Admission Logic

5.1 Round-Robin Selection

`rr_index` in the shared header acts as a shared cursor. Each call to `rr_pick()` atomically reads `rr_index`, increments it (mod `total_words`), releases `rr_mutex`, and then linearly scans forward from that position looking for a word that is neither claimed nor admitted. This gives a fair, starvation-resistant order.

5.2 Atomic Capacity Check

After a word-carrier claims a word it calls `try_admit()`. The function determines `lo = min(arrival_floor, sorting_floor)` and `hi = max(...)`, acquires `floors[lo].mutex` then `floors[hi].mutex` in that order, and only then checks whether both floors are below

`max_words_per_floor`. If yes, both counts are incremented and `admitted=1` is returned before releasing the mutexes. This all-or-nothing check prevents a state where one floor is reserved but the other is full.

5.3 Admission Failure

If either floor is full, both mutexes are released without any change, `word->claimed` is reset to 0, and `header->retry_count` is incremented under `stats_mutex`. The word is then visible to all word-carriers again and may be selected in a future round-robin pass.

6. Character Transportation and Elevators

6.1 Character Task Generation

During shared-memory initialisation each word's `tasks[]` array is populated: `tasks[j]` receives the character, its original index, and the sorting floor as destination. The `src_floor` is set to -1 until a word-carrier admits the word and fills it with the arrival floor, from which point the task is visible to letter-carriers on that floor.

6.2 Delivery Elevator

The elevator process blocks on `request_sem`. When woken it acquires its mutex, moves one floor in the current direction, clears the request for that floor if present, and checks whether to reverse direction. Reversal occurs when there are no pending requests ahead and `current_load` is zero. The elevator compulsorily reverses at floor 0 (direction becomes UP) and floor `num_floors-1` (direction becomes DOWN).

Before boarding, a letter-carrier calls `wait_for_capacity()` which loops checking `current_load < capacity` under the elevator mutex, sleeping 5 ms between attempts if the elevator is full. Once a slot is available `current_load` is incremented atomically inside the mutex. After arrival `current_load` is decremented and `delivery_ops` is incremented under `stats_mutex`.

6.3 Reposition Elevator

Operates with the same direction policy as the delivery elevator. An idle letter-carrier (`idle_ticks` ≥ 5) picks a random destination floor different from its current floor, boards the reposition elevator following the same capacity protocol, and resumes work upon arrival. `reposition_ops` is incremented on each completed trip.

6.4 Idle Letter-Carrier Monitoring

The parent's `monitor_loop()` calls `count_live_lc_on_floor()` on every iteration. This function uses `waitpid(WNOHANG)` to detect exited processes and clears their `lc_states` entry. If a floor's live count reaches zero the parent immediately forks `lc_per_floor` new letter-carrier processes for that floor, appending their PIDs to a dynamically growing list so they are also killed and waited for at shutdown.

7. Sorting Algorithm

7.1 Character Placement

When a letter-carrier delivers a character it calls `place_char()` which holds `word_mutex` and scans `sorting_area[]` for the first slot where `occupied[i]==0` and `fixed[i]==0`. The character is written there and `occupied[i]` and `tasks[j].delivered` are set. Fixed slots are never overwritten.

7.2 Sorting Step

`sort_step()` is the core of each sorting process. For each occupied, non-fixed slot i it calls `find_correct_slot()` to locate the target index j where `original[j] == sorting_area[i]` and `fixed[j]==0`. Three cases follow:

- **Correct position** ($j = i$): `fixed[i] = 1`.
- **Target empty** (`!occupied[j]`): character moved to j, slot i cleared.
- **Target occupied and not fixed**: swap `sorting_area[i]` and `sorting_area[j]`.
- **Target fixed**: skip — another iteration will handle this character.

7.3 Handling Repeated Characters

The naive approach of always returning the first matching index in `original[]` fails for words like `apple` (two `p` characters) because the first match may already be fixed. `find_correct_slot()` uses a two-phase search:

- **Phase 1:** if `original[i] == c` and `!fixed[i]`, return i immediately (already correct).
- **Phase 2:** scan all j where `original[j]==c` and `!fixed[j]` and $j \neq i$. Prefer a slot where `!occupied[j]` (*best_ empty*); fall back to an occupied-but-not-fixed slot (*best_ swappable*). This guarantees that a fixed duplicate never blocks progress.

7.4 Word Completion

After each `sort_step` pass, if all `tasks[i].delivered==1` and all `fixed[i]==1` the word is marked `completed=1`, its contribution to both floor counts is decremented, and `done_count` is incremented under `done_mutex`. The parent's `monitor_loop` exits when `done_count >= total_words`.

8. Termination Detection

Termination uses a layered flag system entirely within shared memory:

- **claimed / admitted flags:** word-carriers exit their loop when `admitted_count >= total_words`, meaning every word has been accepted into the system.
- **completed flag and done_count:** sorting processes set `completed=1` and increment `done_count` when a word is fully sorted. The parent polls `done_count` under `done_mutex`.
- **shutdown flag:** set to 1 by the parent once `done_count >= total_words` (or on SIGINT). All child processes check this flag in their main loops and exit cleanly.

After setting `shutdown=1` the parent sends SIGTERM to every known child PID (both the original `pid_list` and the dynamically added replacement letter-carriers), then calls

`waitpid()` on each one to reap them and prevent zombies. Finally it writes the output file and prints the summary before calling `shm_destroy()` and returning.

On SIGINT the `sigint_handler()` sets `shutdown=1`, kills and waits for all children, destroys shared memory, and calls `_exit(EXIT_FAILURE)`.

9. Output File Generation

`write_output_file()` copies the `words` array into a local buffer, sorts it with `qsort` using `cmp_words()` which compares first by `sorting_floor` then by `word_id`, and writes each entry as:

```
<word_id> <original_word> <sorting_floor>
```

using `fprintf`. The arrival floor is not written. The file is opened with `fopen("w")` so it is created fresh each run. Correctness is guaranteed because `original[]` is never modified after initialisation and `sorting_floor` comes directly from the parsed input.

10. Test Case Scenario

10.1 Input File (`input.txt`)

```
101 apple 2
102 process 0
103 system 1
104 kernel 0
105 thread 2
```

10.2 Command

```
./hw3 -f 3 -w 2 -l 2 -s 2 -c 4 -d 4 -r 2 -i input.txt -o output.txt
```

10.3 Expected Output File (`output.txt`)

```
102 process 0
104 kernel 0
103 system 1
101 apple 2
105 thread 2
```

10.4 Observed Behaviour

At startup the parent spawns $3 \times (2\,\mathrm{WC} + 2\,\mathrm{LC} + 2\,\mathrm{SP}) = 18$ worker processes plus 2 elevator processes, totalling 20 children. Word-carriers claim words in round-robin order and admit them once both floors have room. Letter-carriers pick character tasks concurrently; some use the delivery elevator to cross floors while others place characters directly on the same floor. Sorting processes run in parallel and sort characters as they arrive, even before all characters of a word have been transported. The word `apple`

exercises the duplicate-character path in `find_correct_slot()` because it contains two `p` characters. The system terminates once all five `done_count` flags are set, writes the output file sorted by `sorting_floor` then `word_id`, and prints the summary.

11. Challenges and Solutions

Duplicate characters in sorting
The original `sort_step` always found the first matching index in `original[]`, which could be fixed, permanently blocking a second copy of the same character. The fix was `find_correct_slot()` with its two-phase search: prefer the current slot if already correct, otherwise find the best non-fixed target.

Elevator capacity not enforced
The initial implementation incremented `current_load` unconditionally. The fix adds `wait_for_capacity()` which atomically checks `current_load < capacity` under the elevator mutex and sleeps 5 ms if the elevator is full.

Floor without letter-carriers
If all letter-carriers migrated away from a floor via repositioning, no one would ever pick up tasks there. The `monitor_loop` now calls `count_live_lc_on_floor()` on every tick using `waitpid(WNOHANG)` and spawns replacements when the count drops to zero.

Lock ordering for admission
Two floor mutexes must be acquired simultaneously. Without a consistent order any two word-carriers targeting the same pair of floors in opposite order could deadlock. The solution is to always acquire the mutex for the lower floor index first.

Summary statistics missing
The original `print_summary` only printed three numbers. The header was extended with `retry_count`, `delivery_ops`, and `reposition_ops` protected by `stats_mutex`, and incremented at the appropriate points in `word_carrier.c` and `letter_carrier.c`.

12. Additional Tests

12.1 Stress Test — Many Processes

```
./hw3 -f 5 -w 3 -l 4 -s 3 -c 3 -d 6 -r 3 -i input.txt -o output.txt
```

With 5 floors and tight floor capacity (3 words max) many admission retries are expected. The `retry_count` in the summary confirms round-robin fairness under contention. All words still complete correctly.

12.2 Single Floor

```
./hw3 -f 1 -w 1 -l 2 -s 2 -c 10 -d 4 -r 2 -i input.txt -o output.txt
```

With one floor all `sorting_floor` values must be 0. Letter-carriers place characters directly (same-floor path). Elevators receive no delivery requests and the reposition elevator moves letter-carriers to floor 0 repeatedly, which is a no-op. The system terminates normally.

12.3 Elevator Capacity = 1

```
./hw3 -f 3 -w 2 -l 3 -s 2 -c 5 -d 1 -r 1 -i input.txt -o output.txt
```

With delivery capacity 1, letter-carriers frequently block in `wait_for_capacity()`. The system is slower but still produces correct output, confirming that the capacity enforcement does not cause deadlock.